

Capstone Project

JWT & AuthController Guide

System: Capstone Registry / Evaluation Platform

Audience: Developers, evaluators, documentation readers

Last updated: August 2026

1. What is JWT? (Simple explanation)

JWT means JSON Web Token. Think of it like a theme-park wristband:

- You show your ID once at the gate (email + password).
- The office gives you a wristband (the JWT).
- For every ride (API request), you show the wristband - not your password again.
- Guards check: Is the wristband real? Is it still valid?

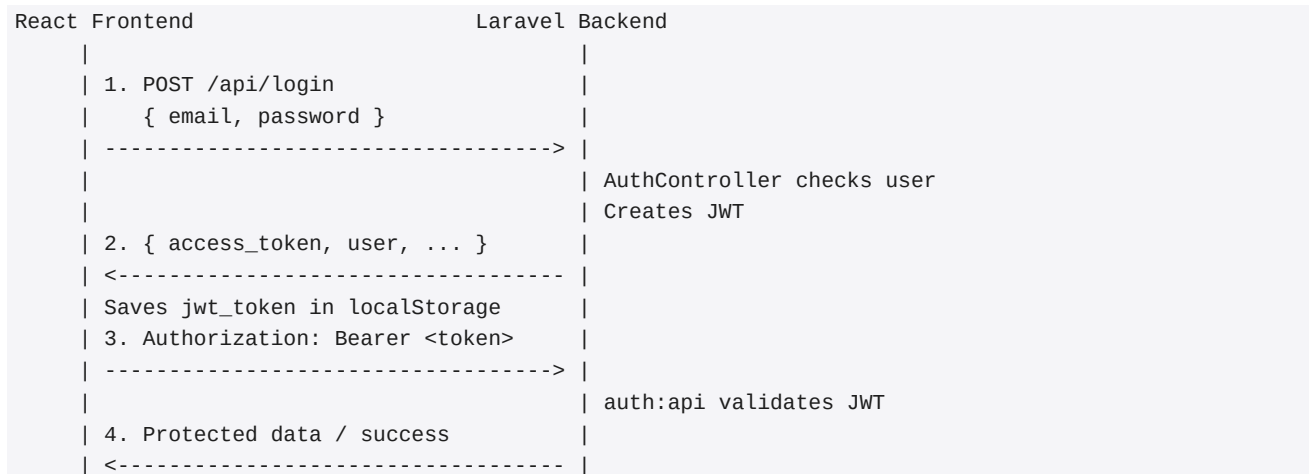
A JWT is a long string with three parts separated by dots:

```
header.payload.signature
```

Header = token type / signing style. Payload = who you are (user_id, email, role) and expiry. Signature = secret stamp from the server (JWT_SECRET) so nobody can fake it.

2. How JWT works in this system

Big picture flow



Where the token lives

- Backend response access_token - JWT after successful login
- Browser localStorage key jwt_token - frontend stores the badge
- Request header Authorization: Bearer ... - every protected API call
- .env JWT_SECRET - secret used to stamp / verify tokens
- .env JWT_TTL - lifetime in minutes (default 60)

Important frontend files

- front-end/src/api/axios.js - attaches Bearer token; refreshes on 401
- front-end/src/services/jwtAuthService.js - login / logout / refresh helpers
- front-end/src/context/AuthContext.js - stores logged-in user for the UI

Important backend files

- AuthController.php - main login for the app
- JwtAuthController.php - register, refresh, alternate JWT API
- AuthSecurity.php - password checks, logout
- AuthUnitHelpers.php - email/password/token helpers
- TblUser.php - implements JWTSubject
- routes/api.php - public vs auth:api protected routes
- config/jwt.php - JWT package settings

3. AuthController - how it works

AuthController is the main authentication door used by the React portal.

Routes

- POST /api/login - log in, return JWT + full user payload
- POST /api/logout - invalidate token / log out
- GET /api/user - return current user (needs valid JWT)

Login steps (AuthController::login)

- 1. Validate input (email format + password required)
- 2. Find user in tbl_users by email
- 3. Security check via AuthSecurity::validateCredentialsForLogin
- 4. Create JWT for that user_id
- 5. Log successful login (UserSessionLogger)
- 6. Return JSON: access_token, Bearer, expires_in, rich user payload, security settings

Why the rich user payload matters

AuthController::userPayload() builds what the frontend needs to know the role, show the correct panel, enforce permissions, and apply evaluation year-level restrictions.

Logout

- Read current JWT, log logout, invalidate/blacklist token
- Frontend deletes jwt_token from localStorage

4. AuthController vs JwtAuthController

Both use JWT. They are NOT session-auth vs token-auth.

- AuthController = main app login (POST /api/login), full portal user payload, used by React login UI
- JwtAuthController = extra JWT tools: register, refresh, thinner login response (id, email, role_id)

Kid version: AuthController = school office that gives a badge AND full record. JwtAuthController = machine that can also make/refresh badges and register users.

Axios uses JwtAuthController refresh when a protected call returns 401.

5. Protected routes

In routes/api.php, public routes include login and some guest browse. Almost everything else is inside middleware auth:api, which means: only continue if the Bearer JWT is valid.

6. Token expiry and refresh

- JWT expires after JWT_TTL minutes (default 60).
- On 401, axios tries POST /api/jwt/refresh once.
- If refresh works: save new token and retry the request.
- If refresh fails: remove token and redirect to /login.

Idle logout (session timeout from security settings) is separate from JWT expiry.

7. Security pieces tied to auth

- Bcrypt password hash - passwords never stored as plain text
- Lockout after failed attempts - stops password guessing
- JWT signature (JWT_SECRET) - stops forged tokens
- Token blacklist on logout - old badge cannot be reused
- Level 1 helpers (UT-101 to UT-106) - email/password/token unit checks

8. Quick FAQ

Q: Does AuthController use JWT? A: Yes. Login returns an access_token JWT.

Q: Why two auth controllers? A: One is main portal login; the other adds register/refresh and a thinner JWT API.

Q: Where is the token saved? A: Browser localStorage.jwt_token.

Q: Send password every request? A: No. Only at login. Later requests send the JWT.

Q: If someone steals my JWT? A: They can act as you until expiry/blacklist - use HTTPS, short TTL, logout, idle timeout.

9. Summary

- 1. User logs in through AuthController (POST /api/login).
- 2. Backend verifies password and issues a JWT.
- 3. Frontend stores it as jwt_token.
- 4. Every protected API call sends Authorization: Bearer <token>.
- 5. Laravel auth:api validates the token and loads the user.
- 6. Logout / expiry / failed refresh removes access.

JWT = temporary signed pass.

AuthController = main door that issues that pass and the full user profile for the portal.